

Architecture Review

YallaJo Platform

DDD | Modular Monolith | Clean Architecture

Gap Analysis - February 20, 2026

Prepared by: AI Architecture Consultant (Oracle)

Executive Summary

As-is, this project is not "DDD + Modular Monolith + Clean Architecture" in practice. It is a set of partially-scaffolded modules with heavy framework coupling through YallaJo.SharedKernel, multiple independent API hosts, and an outbox / integration-events story that is not wired end-to-end.

The biggest structural problem is dependency direction: Domain and Application layers are not framework-agnostic, so the "clean" boundaries are mostly nominal (layering by folders, not by actual dependency inversion).

Severity Overview

Severity	Count	Areas
Critical	4	Clean Architecture violations, SharedKernel coupling, missing inter-module contracts, half-implemented outbox
Major	4	Not a true modular monolith, MediatR in domain, event dispatch risks, missing host
Minor	2	DateTime.UtcNow in domain, repository CQRS separation

Detailed Findings

Finding #1: Not a True Modular Monolith

Major

Each module (Accounts, Auth, Security) has its own Presentation project with a separate Program.cs, making them independent composition roots and effectively separate deployable units.

This means the project pays "distributed system" costs (duplicated hosting config, auth/CORS/versioning, cross-cutting registration, deployment coordination) without gaining clear microservice boundaries (contracts, versioning, independent data ownership).

A true modular monolith has ONE host/bootstrapper that composes all modules as class libraries. Currently, the architecture sits in an awkward middle ground.

Recommendation

Pick one topology:

(A) True modular monolith = single host + modules as class libraries

(B) Microservices = embrace separate deployables + explicit contracts + real integration

Stop mixing signals.

Finding #2: Clean Architecture Violations

Critical

Two critical violations exist:

1. `IUnitOfWork<TContext>` in the Application Abstractions directly references `Microsoft.EntityFrameworkCore.DbContext`. This places an ORM dependency at the Application layer, violating the Dependency Rule.
2. `YallaJo.SharedKernel` bundles Domain + Application + Infrastructure in ONE project and references EF Core, MediatR, and FluentValidation. Since every module's Domain project references SharedKernel, Domain entities transitively depend on all three frameworks.

This means "Clean Architecture" is layering-by-folders, not actual dependency inversion. Domain and Application cannot exist without EF/MediatR, which defeats the core promise.

Recommendation

Split SharedKernel by layer (see Finding #6). Remove the DbContext constraint from IUnitOfWork in the Application layer. The Application layer should only know IUnitOfWork (plain interface); the EF-specific generic UoW stays in Infrastructure.

Finding #3: DDD Concerns (Domain Events, Testability, Consistency)

Major

A. Domain events implement MediatR.INotification (Major)

IDomainEvent directly extends MediatR's INotification, leaking a library dependency into the Domain layer. The domain should define its own event interface; adaptation to MediatR belongs at the Application/Infrastructure edge.

B. DateTime.UtcNow inside domain entities (Minor)

UserEmail.MarkAsVerified() uses DateTime.UtcNow directly. An IDateTimeProvider already exists in SharedKernel but is not used. This makes domain logic non-deterministic under tests.

C. Domain events dispatched AFTER SaveChangesAsync (Major)

The UnitOfWork dispatches domain events after the DB commit succeeds. This prevents publishing events for rolled-back transactions (good), but if event handlers fail, the DB commit already happened with no retry mechanism (bad). Without an outbox for domain event side-effects, this creates an inconsistency window.

D. Domain vs Integration events not connected (Major)

Both IDomainEvent and IIntegrationEvent are defined, but only domain events actually flow. Integration events and the outbox are dead abstractions.

Recommendation

- A. Make IDomainEvent a plain interface; create a MediatR adapter in Infrastructure.
- B. Inject IDateTimeProvider or pass 'now' from the Application layer.
- C. Consider persisting domain event side-effects in the outbox (same transaction), or accept the consistency trade-off and document it.
- D. Wire domain events to produce integration events for cross-module communication.

Finding #4: No Inter-Module Communication

Critical

There is zero mechanism for modules to communicate. When a user registers in the Accounts module and the Auth module needs to create credentials, there is no path.

No integration events are published between modules, no shared contracts or public APIs exist, and no event bus (even in-process) is configured.

For a modular monolith, this is a fundamental gap that will block any real feature requiring cross-module coordination.

Recommendation

Define a small "module contract" surface per module (commands / queries / events). Recommended approach:

- Synchronous: Module exposes an Application-layer interface that other modules call
- Asynchronous (preferred): In-process integration event bus + explicit public contracts

Prohibit direct cross-module Domain layer references.

Finding #5: Outbox Pattern is Half-Implemented

Critical

OutboxMessage.Create(IntegrationEvent) and OutboxProcessor exist in SharedKernel, but:

- The Accounts module never creates integration events
- No outbox records are ever persisted
- OutboxProcessor resolves DbContext generically via GetRequiredService<DbContext>(), which is ambiguous when multiple modules have their own DbContext
- No outbox table configuration exists in any module's DbContext

This is architecture theater: the abstractions exist but nothing is wired end-to-end.

Recommendation

Either:

(A) Complete it: Persist integration events in the SAME transaction as aggregate changes, configure outbox table per module context, run a processor per context

(B) Remove it: Delete the dead code to reduce false architecture signals. Re-introduce when actually needed.

Do not leave half-implemented patterns.

Finding #6: SharedKernel is a Kitchen-Sink

Critical

YallaJo.SharedKernel contains Domain abstractions, Application abstractions, AND Infrastructure implementations (EfRepository, UnitOfWork, OutboxProcessor, SpecificationEvaluator) in a single project.

Its csproj depends on:

- MediatR 14.0.0
- Microsoft.EntityFrameworkCore 9.0.13
- Microsoft.EntityFrameworkCore.Relational 9.0.13
- FluentValidation 12.1.1
- Microsoft.Extensions.Hosting.Abstractions 10.0.3

Every module's Domain project that references SharedKernel transitively inherits ALL of these. A Domain layer that depends on EF Core is not a clean domain.

Recommendation

Split into separate projects:

SharedKernel.Domain

- > BaseEntity, ValueObject, IAggregateRoot, IDomainEvent, DomainException
- > ZERO external dependencies (no MediatR, no EF)

SharedKernel.Application.Abstractions

- > ICommand/IQuery, IUnitOfWork, IRepository, Result, ISpecification
- > NO EF types (no DbContext constraint)

SharedKernel.Infrastructure.EfCore

- > UnitOfWork<T>, EfRepository, SpecificationEvaluator
- > References EF Core here only

SharedKernel.Infrastructure.Messaging (optional)
-> MediatR adapters for domain/integration events

Modules reference ONLY what they need at each layer.

Finding #7: Missing Host / Bootstrapper

Major

There is no single composition root that composes all modules. Key gaps:

- No shared middleware, authentication, or authorization pipeline
- No central place for cross-cutting behaviors (validation, logging, performance)
- No background job orchestration (outbox processor per module)
- No database migration strategy (who runs migrations? per context? centrally?)
- No API gateway or reverse proxy for unified routing

Each module's Presentation project duplicates the full ASP.NET Core setup.

Recommendation

Create one host project (e.g., YallaJo.Api) that:

- Calls each module's AddXxxApplication() and AddXxxInfrastructure() extensions
- Owns shared middleware, auth, CORS, versioning
- Registers cross-cutting MediatR behaviors
- Hosts background jobs (outbox processors)
- Manages migration execution strategy (per-context, centrally triggered)

Modules become class libraries that expose registration and endpoint mapping methods.

Finding #8: Repository CQRS Separation

Minor

IUserRepository extends both IWriteRepository<User, Guid> and IReadRepository<User, Guid>, giving command handlers full read+write access.

Query handlers correctly inject IReadRepository<User, Guid> directly (good CQRS hygiene). However, command handlers get IUserRepository which includes read methods they shouldn't need for pure write operations.

Also, the IUserRepository namespace (Accounts.Domain.Interfaces) doesn't match its folder path (Accounts.Domain/Repositories/).

Recommendation

If enforcing strict CQRS boundaries:

- Inject IUserWriteRepository into commands
- Inject IReadRepository<User, Guid> into queries (already done)

If pragmatic CQRS is acceptable, keep combined but be consistent.

Fix the namespace mismatch: Accounts.Domain.Interfaces -> Accounts.Domain.Repositories.

Action Plan (Priority Order)

1. Decide Deployment Topology

Choose single-host modular monolith OR microservices. Stop mixing signals with multiple Program.cs files claiming to be a monolith.

2. Split SharedKernel by Layer

Create SharedKernel.Domain (zero deps), SharedKernel.Application.Abstractions (no EF), SharedKernel.Infrastructure.EfCore (EF here only).

3. Fix UnitOfWork Abstraction

Application layer depends on IUnitOfWork (plain interface). Remove DbContext constraint from Application. EF-specific UoW stays in Infrastructure.

4. Decouple Domain Events from MediatR

IDomainEvent becomes a plain interface. Create a MediatR adapter/dispatcher in Infrastructure.

5. Complete or Delete the Outbox

Either persist integration events transactionally per module and wire the processor, or remove the dead code entirely.

6. Create Single Host (if Monolith)

Build YallaJo.Api as the single composition root. Modules expose Add{Module}() extensions and endpoint mapping.

7. Define Inter-Module Contracts

Each module publishes a contracts surface (commands/queries/events). Prohibit direct cross-module Domain references.

Effort Estimate

Full Cleanup (splits + host + outbox wiring):

Large - 3+ days

Stop the Bleeding (single host + fix transitive deps + remove dead outbox):

Medium - 1-2 days

Risks & Watch-Outs

- ! Splitting SharedKernel will ripple through ALL project references. Do it incrementally and enforce rules with reference constraints or architecture tests.
- ! Publishing domain events after commit without an outbox creates "DB updated but side-effects missing" failures under transient issues. Accept or fix this trade-off explicitly.
- ! If you keep multiple hosts, own it as microservices and add proper contracts, API versioning, and observability. Otherwise you get the worst of both worlds.